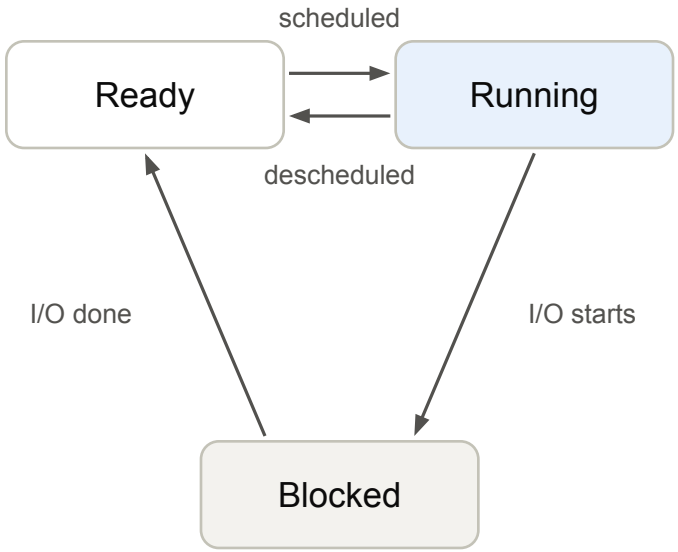
Operating Systems and Parallel Computing

Lecture 11

Prof. Matteo Turilli

Sharing the Processor

- Recall Lecture 4: with **time sharing**, the OS runs a process, stops it, and runs another, many times per second.
- The **mechanism** is the context switch: the OS saves the registers of one process and loads those of another. A timer interrupt gives control back to the OS (Lecture 5).
- The **policy** decides which ready process runs next, and for how long.
- The **scheduler** is the part of the OS that applies the policy. It chooses only among **ready** processes.
- Scheduling matters when more processes are ready than there are cores, e.g., a notebook training a model during a video call.



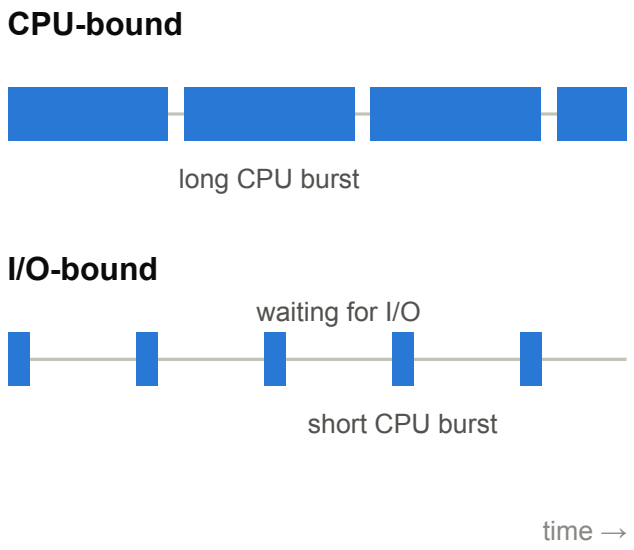
The scheduler moves processes between **Ready** and **Running**.

When Does the Scheduler Run?

- The OS must choose a process to run when:
  - a new process is created: should the parent or the child run?
  - a process exits, or blocks on I/O or on a lock;
  - an I/O interrupt makes a blocked process ready again;
  - a timer interrupt arrives.
- A **non-preemptive** scheduler lets a process run until it blocks, exits or gives up the CPU.
- A **preemptive** scheduler can stop a process at a timer interrupt and run another one. Without preemption, a program stuck in an infinite loop would keep the CPU.
- Linux, macOS and Windows are preemptive. Many early batch systems were not.

CPU-Bound and I/O-Bound Processes

- Processes alternate **CPU bursts** with waits for I/O: disk, network or keyboard.
- A **CPU-bound** process has long CPU bursts, e.g., training a random forest or running a Monte Carlo simulation.
- An **I/O-bound** process computes a little between waits, e.g., downloading files or querying a remote database.
- The length of the CPU burst decides the type, not the length of the wait.
- Running I/O-bound processes as soon as they are ready keeps both the CPU and the devices busy.
- **Question**: our CSV file is in the page cache (Lecture 9, p. 25). Is `pd.read_csv` CPU-bound or I/O-bound?



What Should a Scheduler Optimize?

- Different systems want different things from the scheduler.
- No policy is best for every goal: e.g., batch and interactive goals often conflict.

System	Goals	Examples
All systems	● Fairness: comparable processes get comparable CPU time. ● Balance: keep the CPU and the devices busy.	Every OS
Batch	● Throughput: jobs per hour. ● Turnaround time: from submission to completion. ● CPU utilization.	Payroll, nightly ETL, end-of-day risk at a bank, retraining models
Interactive	● Response time: react to requests quickly. ● Proportionality: simple requests should feel fast.	Jupyter, dashboards, web servers
Real time	● Meet deadlines. ● Predictability: avoid jitter.	Video calls, car brakes, factory robots

Measuring a Scheduler

- We compare policies on a simple **workload** of jobs (processes) and then relax its assumptions one at a time.
- **Turnaround time**: from arrival to completion.

  $T_{turnaround} = T_{completion} - T_{arrival}$
- **Response time**: from arrival to the first time the job runs.

  $T_{response} = T_{first\ run} - T_{arrival}$
- Example at a supermarket:
  - **Turnaround**: the time from joining the queue to leaving the shop.
  - **Response**: the time until a cashier starts serving you.
- Batch systems care about turnaround, interactive systems about response.

Starting assumptions

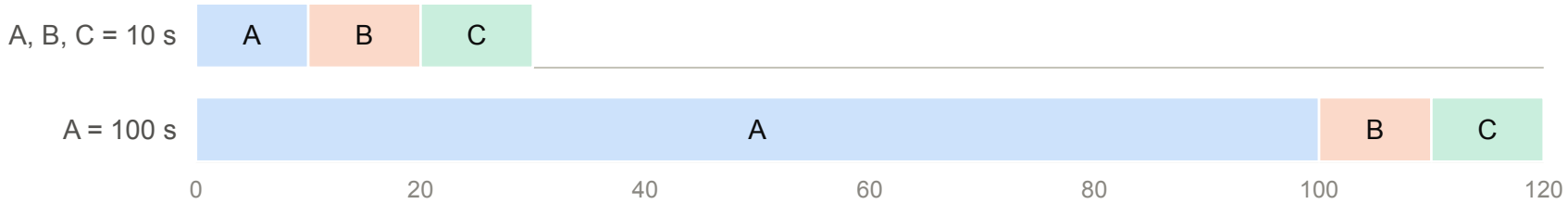
- We start with an oversimplified problem
- Progressively, we relax the following assumptions
- We obtain more and more complex scheduling policies

Assumptions:

1. Each job runs for the same time (slide 7).
2. All jobs arrive at the same time (slide 9).
3. Once started, a job runs to completion (slide 9).
4. Jobs use only the CPU, no I/O (slide 13).
5. The scheduler knows each run time (slide 14).

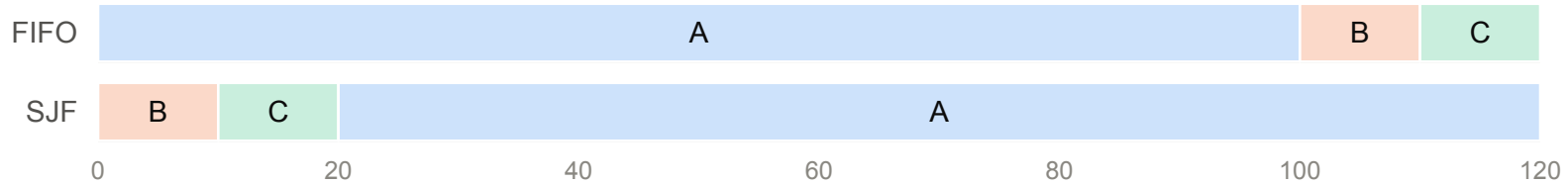
First In, First Out (FIFO)

- **FIFO**, or first come first served, runs jobs in order of arrival, each to completion. It is simple to implement.
- A, B, and C arrive together at T_0 and need 10s each:
  - Average turnaround (10 + 20 + 30) / 3 = **20s**,
  - $((T_{10}-T_0)+(T_{20}-T_0)+(T_{30}-T_0))/3$ = the time spent by each job in the system on average
- If A needs 100s: (100 + 110 + 120) / 3 = **110s**. B and C wait 100s for a 10s job.
- This is the **convoy effect**: short jobs queue behind a long one, like a single checkout where the customer in front of you has three full carts.



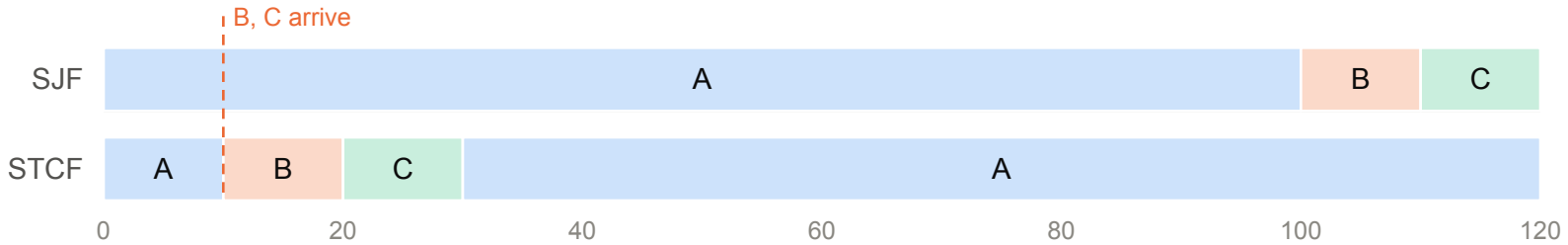
Shortest Job First (SJF)

- **SJF** runs the shortest job first, then the next shortest, and so on.
- A needs 100s, B and C need 10s: SJF runs B, C, then A. Average turnaround (10 + 20 + 120) / 3 = **50s**, against 110 s with FIFO.
- When all jobs arrive together, SJF gives the lowest average turnaround:
  - If four jobs of length a, b, c, d run in that order, they finish at a, a+b, a+b+c, and a+b+c+d.
  - The average is (4a + 3b + 2c + d) / 4: the first job counts four times.
- The idea comes from operations research in the 1950s. Supermarkets use it for express checkout lanes.
- SJF needs run times in advance. Batch systems can often estimate them, e.g., the same 1,000 insurance claims every night.
- This works only because all the jobs arrive together and their lengths are known. That is why slide 9 (late arrivals) and slide 14 (unknown run times) need other policies.



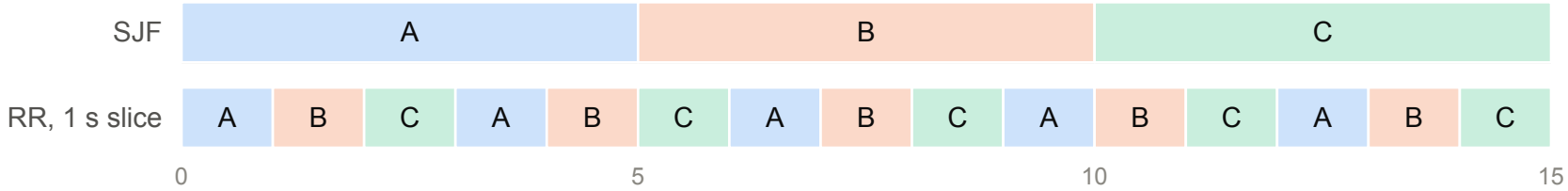
Late Arrivals Need Preemption

- Relax assumptions 2 and 3: A arrives at t = 0, while B and C arrive at t = 10.
- SJF is **non-preemptive**: B and C wait for A to finish. Average turnaround (100 + 100 + 110) / 3 = **103.3s**.
- **Shortest Time-to-Completion First (STCF)**, or preemptive SJF, stops A when B and C arrive, so A no longer runs to completion (assumption 3). It always runs the job with the least time left.
- STCF: (120 + 10 + 20) / 3 = **50s**. It needs the timer interrupt and the context switch from Lecture 5.



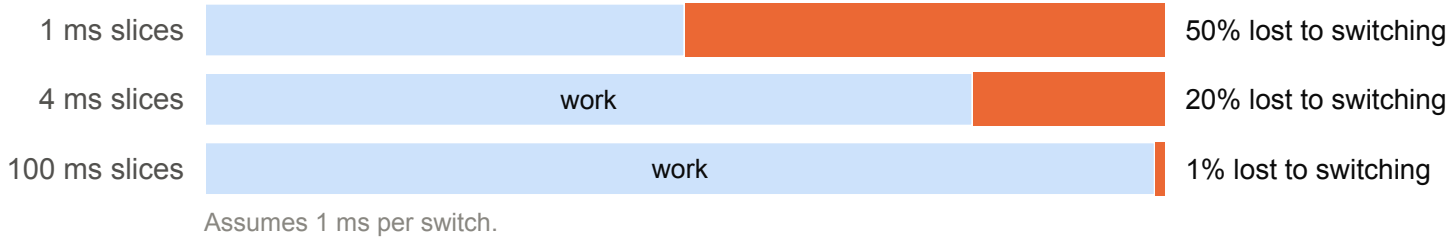
Response Time and Round Robin

- Interactive users care about **response time**: how long until their job first runs.
- A, B and C arrive together and need 5s each. SJF runs them one after the other: average response time is (0 + 5 + 10) / 3 = **5s**.
- **Round Robin (RR)** runs each job for a predefined **time slice**, or quantum, and then moves it to the back of the queue.
- With 1s slices:
  - average response (0 + 1 + 2) / 3 = **1s**.
  - Average turnaround grows from 10s with SJF to (13 + 14 + 15) / 3 = **14s**.
  - RR stretches the time each job stays in the system by interleaving their execution.



How Long Should a Time Slice Be?

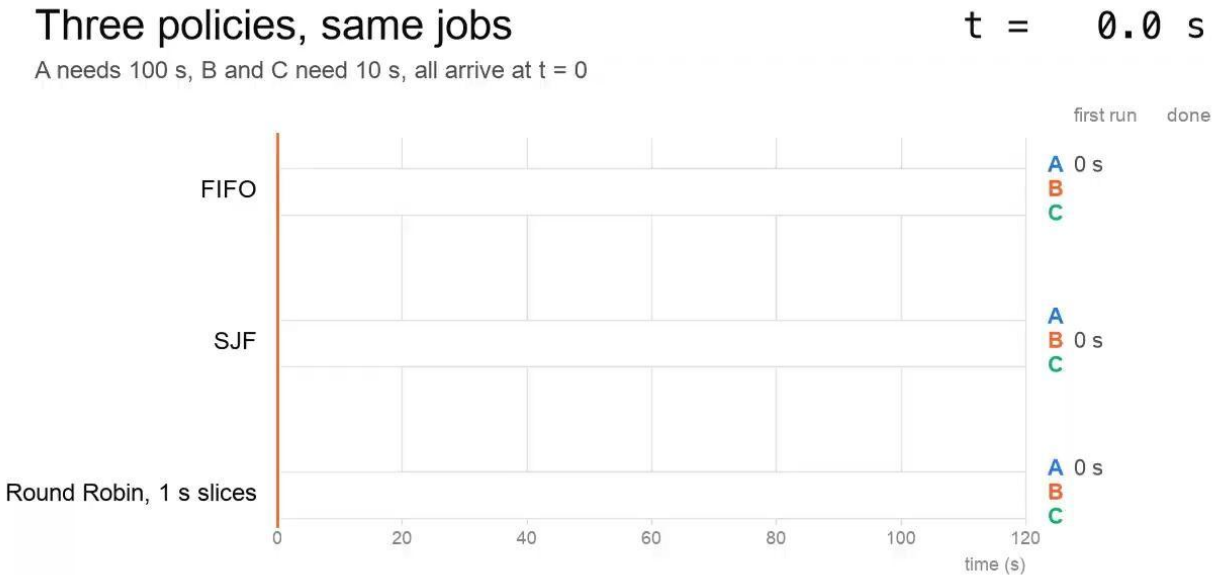
- Each switch costs time: the OS saves and restores registers and runs the scheduler, and the next process starts with cold caches.
- With 4ms slices and 1ms per switch, 1ms in 5 is lost: **20%** of the CPU. With 100ms slices, about **1%**.
- Long slices hurt response time: with 50 ready processes and 100ms slices, the last one can wait **5s**.
- This is **amortization**: pay a fixed cost less often, as when we batch system calls (Lecture 5).
- Tanenbaum suggests 20–50ms as a compromise. Linux aims for slices of a few milliseconds.



Assumes 1 ms per switch.

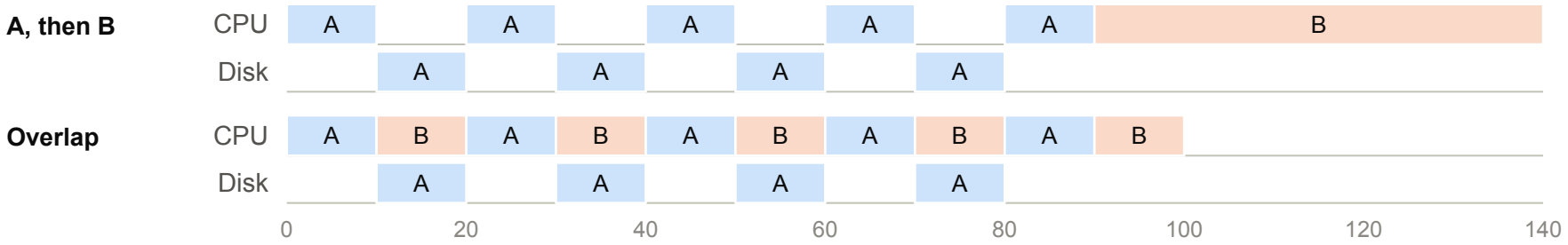
Three Policies, Same Jobs

- A needs 100s, B and C need 10s, and all three arrive together. RR uses 1s slices.
- Watch when each job first runs and when it finishes.
- Interactive version: mturilli.github.io/ospc-materials/animations/scheduling-policies.html



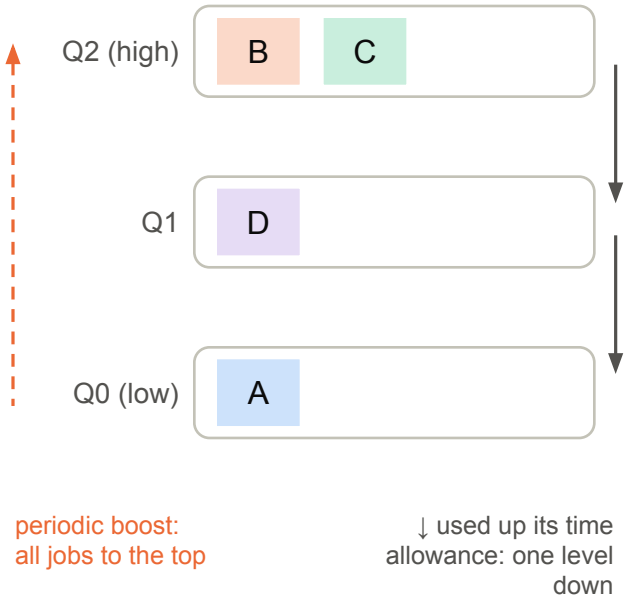
Overlapping I/O and Computation

- Relax assumption 4: jobs do I/O. A job waiting for I/O is blocked, so the CPU can run another job.
- A and B need 50ms of CPU each. A computes in five 10ms bursts, with a 10ms disk read between them. B only computes.
- Running A and then B leaves the CPU idle during every read of A: 140ms in total.
- If each 10ms burst of A counts as a short job, STCF runs A as soon as its read completes and fills the gaps with B: 100ms.
- I/O-bound and interactive jobs get the CPU quickly, and the disk stays busy.



No Oracle: Learning From the Past

- Relax assumption 5: a general-purpose OS does not know how long a job will run.
- The **multi-level feedback queue (MLFQ)** watches how jobs behave and adjusts their priority:
  - Higher-priority jobs run first. Jobs with the same priority share the CPU in round robin.
  - A new job starts at the top.
  - A job that uses up its time allowance at a level moves down one level.
  - Every so often, all jobs move back to the top, so long jobs do not **starve**.
- Short and interactive jobs finish near the top, as with SJF. Long CPU-bound jobs sink and share what is left.
- BSD Unix, Solaris and Windows use forms of MLFQ.



MLFQ in Motion

- First: a long job A, a short job B and an interactive job C. Which queue does each end up in?
- Then: short requests keep Q2 busy. What happens to A without a boost?
- Interactive version: mturilli.github.io/ospc-materials/animations/mlfq.html

